

RAG with Amazon Bedrock Knowledge Bases

Evaluation & demonstration — grounded question answering over an AWS whitepaper corpus

Experiment 1 Setup

Knowledge base ID	XL69XTCTJL
Region	us-east-1
Generation model	Claude Haiku 4.5 (Bedrock inference profile)
Retrieval API	RetrieveAndGenerate (answers) + Retrieve (accuracy measurement), top-k = 5
Chunking strategy	Fixed-size, ~300 tokens, ~20% overlap
Document corpus	AWS whitepapers: Well-Architected pillars (Security, Cost Optimization, Reliability, Performance Efficiency, Operational Excellence, Sustainability, Serverless Lens), Cloud Adoption Framework overview, AWS overview, and Agentic AI Lens

Note: retrieval accuracy is measured with the Retrieve API rather than answer citations. Because a custom “answer only from the retrieved context” prompt is used to enforce the “I don’t know” behaviour, the RetrieveAndGenerate citations field is often empty even when chunks are used, so it is not a reliable accuracy signal.

First Experiment: Answerable questions (grounded in the documents)

ANSWERABLE · GROUNDED IN DOCUMENTS

What is the AWS shared responsibility model, and which security responsibilities fall to the customer versus AWS?

MODEL RESPONSE

The AWS Shared Responsibility Model divides security responsibilities between AWS and its customers:

AWS Responsibility: "Security of the Cloud"

AWS is responsible for protecting the infrastructure that runs all AWS Cloud services, including hardware, software, networking, and facilities; physical and environmental controls; and infrastructure-layer security.

Customer Responsibility: "Security in the Cloud"

Customer responsibilities vary based on the AWS services selected.

For Infrastructure as a Service (IaaS) like Amazon EC2: guest operating system management (updates and security patches), application software, security group (firewall) configuration, and all necessary security configuration and management tasks.

For abstracted services (like Amazon S3 and Amazon DynamoDB): data management (including encryption), asset classification, and IAM permissions and access control.

Shared Controls

Some controls are shared in different contexts — patch management, configuration management, and awareness and training — where AWS handles the infrastructure side and the customer handles their own guest OS, applications, and staff.

RETRIEVED CHUNKS (RETRIEVE API — GROUND TRUTH)

Rank	Source document	Score	Chunk snippet
1	wellarchitected-security-pillar.pdf	0.721	"For AWS managed services operating on multi-tenant architectures (such as Amazon ElastiCache Serverless, Amazon DynamoDB, and Amazon S3), patch manage..."
2	wellarchitected-security-pillar.pdf	0.689	"this differentiation of responsibility is commonly referred to as Security "of" the Cloud versus Security "in" the Cloud."
3	wellarchitected-security-pillar.pdf	0.668	"For abstracted services, such as Amazon S3 and Amazon DynamoDB, AWS operates the infrastructure layer, the operating system, and platforms, and custom..."

ANSWERABLE · GROUNDED IN DOCUMENTS

What design considerations does the Agentic AI Lens describe for building agentic AI workloads on AWS?

MODEL RESPONSE

What makes agentic AI unique

Agentic AI systems introduce architectural dimensions that require specialized design considerations:

Agents reason, not only respond: a single request can trigger multiple LLM inference calls, tool invocations, memory retrievals, and inter-agent communications — iterative reasoning loops that add latency, cost, and failure surface.

Agents act autonomously: agents invoke tools, modify data, and interact with external systems without explicit human instruction at each step, demanding purpose-built security controls, permission boundaries, and human-oversight patterns.

Key design principles

- Foundational infrastructure supporting agent compute and memory
- Orchestration patterns coordinating multi-agent workflows
- Operational practices keeping agents reliable, secure, and cost-effective in production

The lens provides guidance from agent prototypes through production-grade agent systems, addressing reliability, security, and cost-effectiveness at scale.

RETRIEVED CHUNKS (RETRIEVE API — GROUND TRUTH)

Rank	Source document	Score	Chunk snippet
1	agentic-ai-lens.pdf	0.745	<i>“The AWS Well-Architected Agentic AI Lens answers this question by extending the AWS Well-Architected Framework with best practices specific to the des...”</i>
2	agentic-ai-lens.pdf	0.713	<i>“The goal isn't to remove human judgment but to apply it where it matters most. Cost and sustainability require intentional design: the token-based cos...”</i>
3	agentic-ai-lens.pdf	0.690	<i>“AWS Well-Architected Framework Agentic AI Lens. Copyright 2026 Amazon Web Services, Inc. and/or its affiliates.”</i>

Questions the model should decline (not in the documents)

CORRECTLY REFUSED • NOT IN DOCUMENTS

What is the exact hourly price of a t3.medium EC2 instance in the us-east-1 Region?

MODEL RESPONSE

I don't know based on the provided documents.

The search results discuss Amazon EC2 pricing models and instance types in general terms, but they do not contain the specific hourly price for a t3.medium instance in the us-east-1 region.

RETRIEVED CHUNKS (RETRIEVE API — GROUND TRUTH)

Rank	Source document	Score	Chunk snippet
1	aws-overview.pdf	0.502	"You pay a very low rate for the compute capacity you actually consume. For a more detailed description, refer to Amazon EC2 pricing. Amazon EC2 instan..."
2	aws-overview.pdf	0.481	"R7g Instances, powered by the latest generation AWS Graviton3 processors, provide the best price performance in Amazon EC2 for memory-intensive worklo..."
3	wellarchitected-cost-optimization-pillar.pdf	0.479	"Perform an analysis on the workload using the recommendations in cost management tools and apply business rules to those recommendations"

Why this is correct: Related pricing content was retrieved (highest score 0.502), but no chunk contained the specific figure — the model correctly declined to invent a number rather than hallucinate a price.

CORRECTLY REFUSED • NOT IN DOCUMENTS

Who is the current CEO of AWS?

MODEL RESPONSE

I don't know based on the provided documents.

The search results do not contain information about the current CEO of AWS. They only mention that a CEO is a common stakeholder in the Business perspective of the AWS Cloud Adoption Framework, but do not identify who currently holds that position.

RETRIEVED CHUNKS (RETRIEVE API — GROUND TRUTH)

Rank	Source document	Score	Chunk snippet
1	wellarchitected-security-pillar.pdf	0.445	"Security Solutions Architect, Amazon Web Services; Luis Pastor, Senior Security Solutions Architect; Colin Igbokwe, Senior Security S"
2	aws-overview.pdf	0.441	"New services can be provisioned quickly, without the upfront fixed expense. This allows enterprises, start-ups, small and medium-sized businesses"
3	overview-aws-cloud-adoption-framework.pdf	0.441	"Each perspective comprises a set of capabilities that functionally related stakeholders own or manage in your cloud transformation journey."

Why this is correct: All retrieved chunks scored below 0.45 and none held the answer; the model refused and explained why, rather than guessing a name.

RAG vs. a direct LLM call

COMPARISON • SAME QUESTION, BOTH WAYS

When was the Security Pillar of the AWS Well-Architected Framework published?

RAG RESPONSE (GROUNDED IN THE KNOWLEDGE BASE)

Based on the search results, the Security Pillar AWS Well-Architected Framework publication date is **November 6, 2024**.

RETRIEVED CHUNKS (RETRIEVE API — GROUND TRUTH)

Rank	Source document	Score	Chunk snippet
1	wellarchitected-security-pillar.pdf	0.756	"AWS Well-Architected Framework. Security Pillar. Copyright 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved."
4	wellarchitected-security-pillar.pdf	0.746	"Security Pillar - AWS Well-Architected Framework. Publication date: November 6, 2024"
5	wellarchitected-security-pillar.pdf	0.675	"Resources ... (table of contents entry)"

DIRECT LLM RESPONSE (NO KNOWLEDGE BASE)

The Security Pillar was part of the original AWS Well-Architected Framework, which was first published in **2015**. AWS has continued to update the framework over time (notable revisions around 2018 and 2021), and the Security Pillar has remained a core component throughout.

Result: RAG returned the correct, document-grounded date (November 6, 2024) with the exact source chunk visible at rank 4. The direct call — with no access to the document — confidently answered 2015, a plausible but incorrect date. This is a clear case of retrieval grounding preventing a hallucination.

Edge case — over-refusal on a partially-covered question (fixed-size)

Not every refusal is a correct one. This multi-part question is largely answerable from the corpus — Lambda, Fargate, and ECS are all well covered — but EKS appears only in a passing mention. The relevant container chunks were retrieved (EKS even surfaced at rank 5), yet under the strict “answer only from the retrieved context” prompt the model declined the whole question rather than answer the three well-covered services and flag EKS as thin. This is an over-refusal: a retrieval that was good enough, blocked at the generation step by a conservative grounding instruction. It is the specific failure that motivated the hierarchical-chunking experiment that follows.

FIXED-SIZE KB (XL69XTCTJL) · OVER-REFUSAL

Compare Lambda, Fargate, ECS and EKS

MODEL RESPONSE

Sorry, I am unable to assist you with this request.

RETRIEVED CHUNKS (RETRIEVE API — GROUND TRUTH)

Rank	Source document	Score	Chunk snippet
1	aws-overview.pdf	0.539	“Amazon ECS has two modes: Fargate launch type and EC2 launch type. With Fargate launch type, all you have to do is package your application in contain...”
3	aws-overview.pdf	0.502	“managed batch processing at any scale. Containers: Amazon Elastic Container Service (Amazon ECS) — highly secure, reliable, and scalable way to run co...”
5	wellarchitected-reliability-pillar.pdf	0.467	“Amazon Elastic Container Service (Amazon ECS), and Amazon Elastic Kubernetes Service (Amazon EKS) can help you get started faster with containers.”

(a) Retrieval accuracy — did the right chunks come back?

For each question the Retrieve API returned the top 5 chunks. For answerable questions, a hit means the chunk containing the answer came from the expected source document and appeared in the returned set (its rank is noted). For questions outside the corpus, a correct result means no strongly-relevant chunk was returned and the model declined to answer.

#	Test question	Category	Right chunks retrieved?	Top score
1	AWS shared responsibility model	Answerable	✓ Yes — security-pillar @ rank 1	0.721
2	Cost optimization design principles	Answerable	✓ Yes — cost-optimization-pillar @ rank 1 *	0.671
3	AWS CAF perspectives	Answerable	✓ Yes — cloud-adoption-framework @ rank 1	0.714
4	Reliability definition & design principles	Answerable	✓ Yes — reliability-pillar @ rank 1	0.630
5	Agentic AI Lens design considerations	Answerable	✓ Yes — agentic-ai-lens @ rank 1	0.745
6	Chocolate-chip cookie recipe	Out-of-domain	✓ No relevant chunk — model refused	0.369
7	Current CEO of AWS	Not in docs	✓ No relevant chunk — model refused	0.445
8	Hourly price of a t3.medium instance	Not in docs	✓ Related but no answer — model refused	0.502
9	Max S3 buckets per account	Not in docs	✓ No relevant chunk — model refused	0.439
10	Company employee data-retention policy	Out-of-domain	✓ No such document — model refused	0.431
E	Compare Lambda, Fargate, ECS and EKS	Partial coverage	~ Partial — ECS/Fargate chunks returned and EKS appears at rank 5, but the model over-refused	0.539
C	Security Pillar publication date (RAG vs direct)	Comparison	✓ Yes — security-pillar @ rank 1	0.756

* For the cost-optimization question the correct document ranked #1, but several top chunks were front-matter / table-of-contents fragments; the answer was still correct, which suggests the fixed-size chunker picked up boilerplate pages and is a candidate for a cleaner parsing / chunking pass.

~ Row E is neither a clean hit nor a correct refusal: retrieval returned relevant container chunks (and EKS at rank 5), but the strict grounding prompt caused the model to over-refuse a partially-answerable question. Counted separately from the 5 / 5 answerable and 5 / 5 out-of-scope results.

Summary

- Answerable set: 5 / 5 retrieved the correct source document at rank 1, with top-chunk scores of 0.63 – 0.75.
- Out-of-scope set: 5 / 5 were handled correctly — top scores stayed in the 0.37 – 0.50 range and the model refused to answer in every case.
- Clean separation: the lowest answerable score (0.630) sits well above the highest out-of-scope score (0.502) — a ~0.13 gap with no overlap, showing retrieval reliably distinguishes in-corpus from out-of-corpus questions.
- Grounding value: the comparison question shows RAG returning the correct document-sourced date where the bare model produced a confident but wrong answer.
- Over-refusal edge case (row E): on a partially-covered question the relevant chunks were retrieved but strict grounding caused a refusal — a generation-step limitation, not a retrieval miss, later resolved by hierarchical chunking.

Second experiment — hierarchical chunking

The front-matter fragments seen with fixed-size chunking, and cases where the model refused a partially-covered question, motivated testing a second knowledge base (HBDAW1STS4) built with hierarchical chunking. Here documents are split into large parent blocks (~1500 tokens) containing small child blocks (~300 tokens); only the children are embedded and searched, but the matching parent block is returned to the model — so retrieval stays precise while the model receives more surrounding context. The example below previously produced a blunt refusal under fixed-size chunking and is now answered in full. The Reliability-pillar question is shown in the head-to-head comparison that follows.

Experiment 2 Setup

Knowledge base — fixed-size	XL69XTCTJL
Knowledge base — hierarchical	HBDAW1STS4
Region	us-east-1
Generation model	Claude Haiku 4.5 (Bedrock inference profile)
Retrieval API	RetrieveAndGenerate (answers) + Retrieve (accuracy measurement), top-k = 5
Chunking — fixed-size KB	Fixed-size, ~300 tokens, ~20% overlap
Chunking — hierarchical KB	Hierarchical, parent ~1500 tokens / child ~300 tokens (children are embedded and searched; the matching parent block is returned to the model)
Document corpus	Same corpus for both knowledge bases — AWS whitepapers: Well-Architected pillars (Security, Cost Optimization, Reliability, Performance Efficiency, Operational Excellence, Sustainability, Serverless Lens), Cloud Adoption Framework overview, AWS overview, and Agentic AI Lens

Compare Lambda, Fargate, ECS and EKS

MODEL RESPONSE

Comparison of Lambda, Fargate, ECS, and EKS

Based on the provided documents, here's how these AWS compute services compare:

AWS Lambda

- **Type:** Serverless compute service
- **Key feature:** Run code without provisioning or managing servers
- **Pricing:** Pay only for compute time consumed; no charge when code is not running
- **Use cases:** Microservices, event-driven applications
- **Management:** Zero administration — Lambda handles everything required to run and scale code with high availability

AWS Fargate

- **Type:** Serverless compute engine for containers
- **Key feature:** Run containers without managing servers or clusters
- **Management:** No need to provision, configure, or scale VM clusters
- **Control level:** Lower operational overhead — you specify CPU, memory, networking, and IAM policies

Amazon ECS (Elastic Container Service)

- **Type:** Container orchestration service
- **Launch types:** Fargate launch type (serverless) and EC2 launch type (you manage server clusters with more granular control)
- **EC2 mode control:** You're responsible for provisioning, patching, and scaling clusters; you decide server types and container placement

Amazon EKS (Elastic Kubernetes Service)

- **Type:** Fully managed Kubernetes service
- **Key feature:** Kubernetes-based container orchestration
- **Use cases:** Applications requiring Kubernetes orchestration

Summary: Lambda is for serverless code execution, Fargate removes infrastructure management for containers, ECS offers container orchestration with optional infrastructure control, and EKS provides managed Kubernetes.

RETRIEVED CHUNKS (RETRIEVE API — PARENT BLOCKS)

Rank	Source document	Score	Chunk snippet
1	aws-overview.pdf	0.552	"Start with your source code or a container image. AWS App Runner automatically builds and deploys the web application and load balances traffic with e..."
2	wellarchitected-performance-efficiency-pillar.pdf	0.482	"icmpid=docs_homepage_featuredsvcs https://aws.amazon.com/ecs/ https://aws.amazon.com/eks/ https://aws.amazon.com/batch/"
3	aws-overview.pdf	0.480	"For general information, see Compute on AWS. Topics: Compare AWS compute services; Amazon EC2; Amazon EC2 Auto Scaling; Amazon EC2 Image Builder"

Fixed-size vs. hierarchical — the same question, two outcomes

Identical question, model, and grounding prompt run against both knowledge bases. The only variable is the chunking strategy.

SYNTHESIS QUESTION · SAME PROMPT FOR BOTH KBS

According to the Reliability pillar, how do the foundational requirements, change management, and failure management best-practice areas work together to achieve reliability, and what specific design principles support each area?

Fixed-size KB

I don't know based on the provided documents.

The search results contain information about foundational requirements and mention that the Reliability pillar includes best practices for design, delivery, and maintenance of AWS environments. However, the documents do not provide a comprehensive explanation of how foundational requirements, change management, and failure management work together as best-practice areas, nor do they detail the specific design principles that support each of these areas.

RETRIEVED CHUNKS (RETRIEVE API)

Rank	Source document	Score	Chunk snippet
1	wellarchitected-reliability-pillar.pdf	0.533	"Understanding availability needs. Foundations. Foundational requirements are those whose scope extends bey"
2	wellarchitected-reliability-pillar.pdf	0.533	"(table-of-contents entry) 267 Reliability Pillar AWS Well-Architected Framework"
3	wellarchitected-reliability-pillar.pdf	0.531	"(table-of-contents entry) Reliability Pillar AWS Well-Architected Framework. Publication date: November 6"
4	wellarchitected-reliability-pillar.pdf	0.530	"https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/back-up-data.html"
5	wellarchitected-reliability-pillar.pdf	0.528	"(table-of-contents entry) 210 REL12-BP03 Test scalability and performance requirements"

Hierarchical KB

Reliability Pillar: Key Components

Based on the AWS Well-Architected Framework documentation, here are the best-practice areas and design principles for the Reliability pillar:

Best-Practice Areas

The reliability pillar includes **three best practice areas**:

1. **Foundations** — an environment where service quotas and network topology accommodate the workload
2. **Change Management** — how to introduce and manage change in your environment, including runbooks and deployment strategies
3. **Failure Management** — backing up data, applying best practices to withstand component failures, and planning for disaster recovery

Design Principles

1. **Automatically recover from failure** — monitor workloads for key performance indicators (KPIs) and run automation when thresholds are breached
2. **Test recovery procedures** — validate recovery strategies and simulate failures to expose and fix failure pathways before real scenarios occur
3. **Scale horizontally to increase aggregate workload availability** — replace large resources with multiple smaller resources to reduce single points of failure

4. **Stop guessing capacity** — monitor demand and utilization, and automate resource addition/removal to maintain optimal levels

5. **Manage change through automation** — make infrastructure changes using automation that can be tracked and reviewed

Foundational Requirements

Foundational requirements extend beyond a single workload and must be in place before architecting any system. In AWS, most are already incorporated or can be addressed as needed, unlike on-premises environments.

RETRIEVED CHUNKS (RETRIEVE API — PARENT BLOCKS)

Rank	Source document	Score	Chunk snippet
1	wellarchitected-reliability-pillar.pdf	0.541	<i>"The guidance, therefore, is to be thorough in applying the standards and considering the appropriate availability target for the entire lifecycle of o..."</i>
2	wellarchitected-serverless-applications-lens.pdf	0.539	<i>"Data protection. Consider enabling API Gateway Access Logs and selectively choose only what you need, since the logs might contain sensitive data"</i>
3	wellarchitected-reliability-pillar.pdf	0.532	<i>"(table-of-contents entry) 208 REL12-BP01 Use playbooks to investigate failures"</i>
4	wellarchitected-reliability-pillar.pdf	0.530	<i>"Change management and operational resilience. Change management includes how to introduce and manage change in your environment. Implementing change r..."</i>

Why they diverged: The scores were nearly identical (~0.53) in both runs, so this was not a ranking problem — it was a context-size problem. Fixed-size retrieval landed on the Foundations intro plus table-of-contents and URL fragments and never surfaced the change-management or failure-management body text, so under the strict “answer only from context” prompt the model refused. Hierarchical retrieval returned larger parent blocks — including the “Change management and operational resilience” body text at rank 4 — giving the model enough connected content to answer all three areas. The same pattern held for the “Lambda, Fargate, ECS and EKS” comparison, which fixed-size declined and hierarchical answered.

Takeaway for chunking strategy (c): fixed-size chunking retrieved the correct document but often in fragments too small to answer synthesis questions that span a section; hierarchical chunking preserved that surrounding context and converted two refusals into complete, grounded answers, at the cost of a second vector store. For this corpus of long, section-structured whitepapers, hierarchical is the better fit for multi-part questions.